

Mahidol University
Faculty of Information
and Communication Technology

Text Editors

ITCS 254 | Lecture 8

Ittipon (Art) Rassameeroj

Agenda



Overview of Text Editors in Linux



Basic Editors: nano and gedit

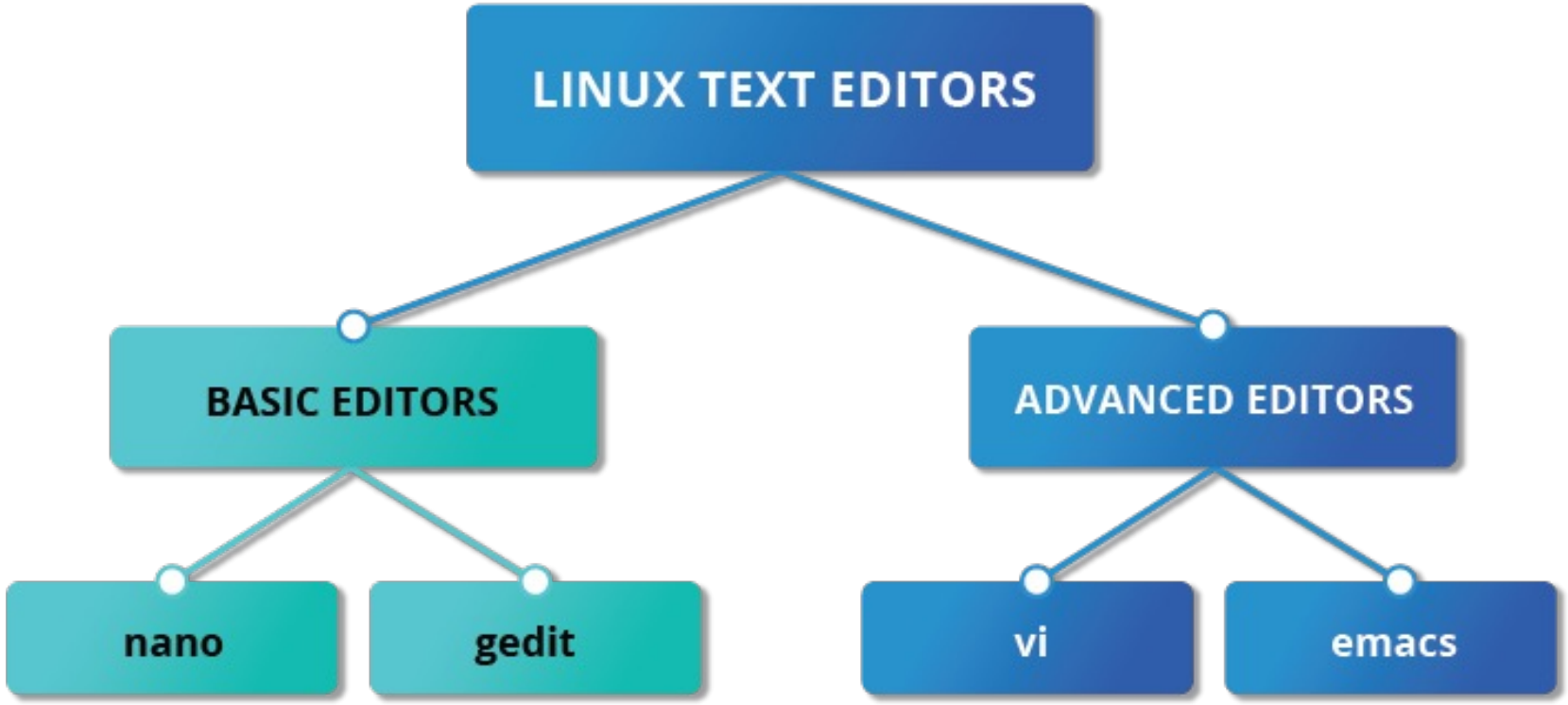


vi Text Editor

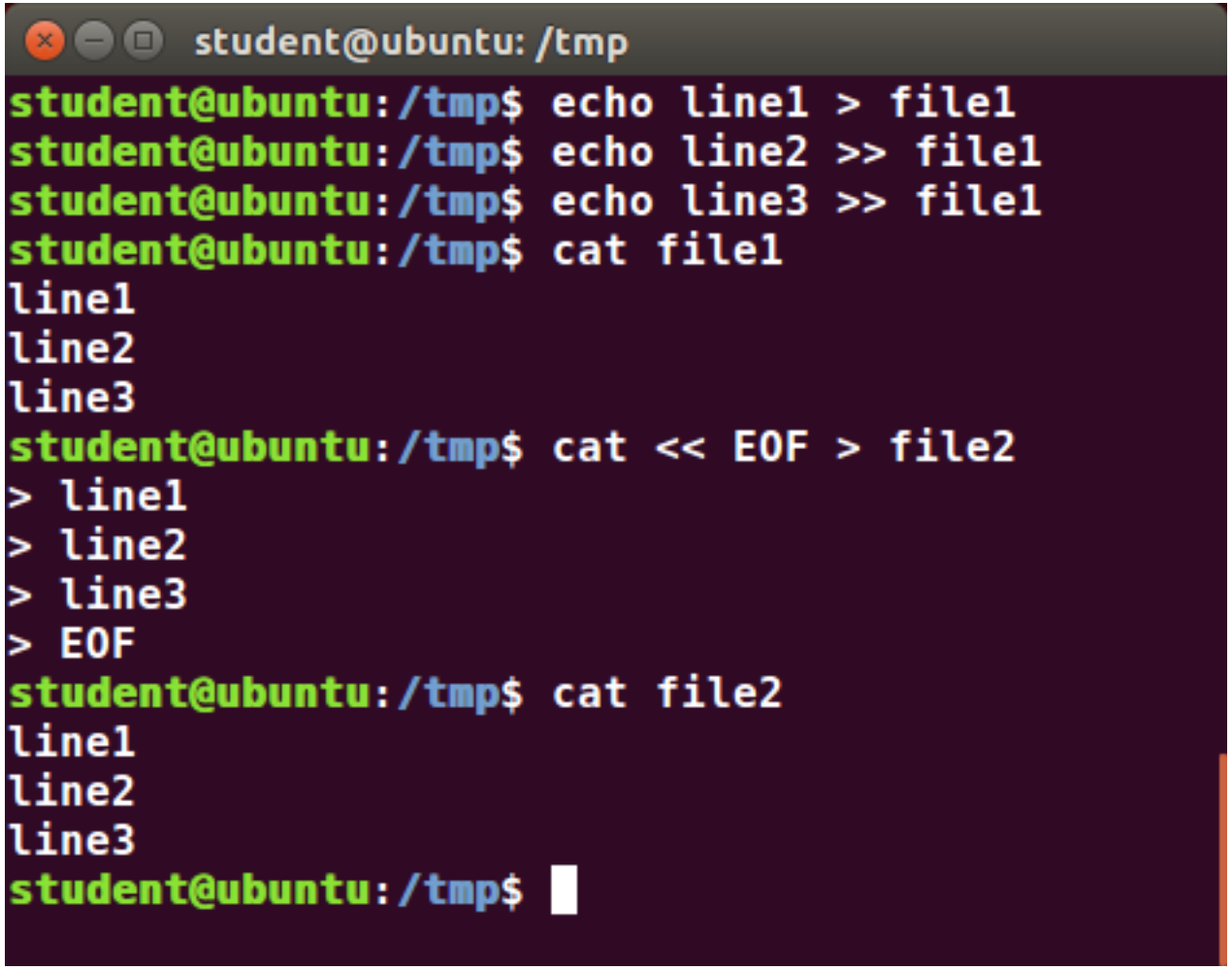


emacs Text Editor

Overview of Text Editors in Linux



Creating Files Without Using an Editor



nano

- nano is easy to use and requires very little effort to learn.
- To open a file, type `nano <filename>` and press Enter. If the file does not exist, it will be created.
- nano provides a two-line shortcut bar at the bottom of the screen that lists the available commands.
  - `CTRL-G` Display the help screen.
  - `CTRL-O` Write to a file.
  - `CTRL-X` Exit a file.
  - `CTRL-R` Insert contents from another file to the current buffer.
  - `CTRL-C` Show cursor position.

nano

```
  GNU nano 2.9.8                        lab_miscdev.h

#ifndef _LAB_CHAR_H
#define _LAB_CHAR_H

#include <linux/module.h>
#include <linux/fs.h>
#include <linux/uaccess.h>
#include <linux/sched.h>
#include <linux/init.h>
#include <linux/slab.h>
#include <linux/device.h>
#include <linux/miscdevice.h>
#include <linux/oom.h>

#define MYDEV_NAME "mycdrv"

static struct device *my_dev;
static char *ramdisk;
static size_t ramdisk_size = (16 * PAGE_SIZE);

static const struct file_operations mycdrv_fops;

/* generic entry points */

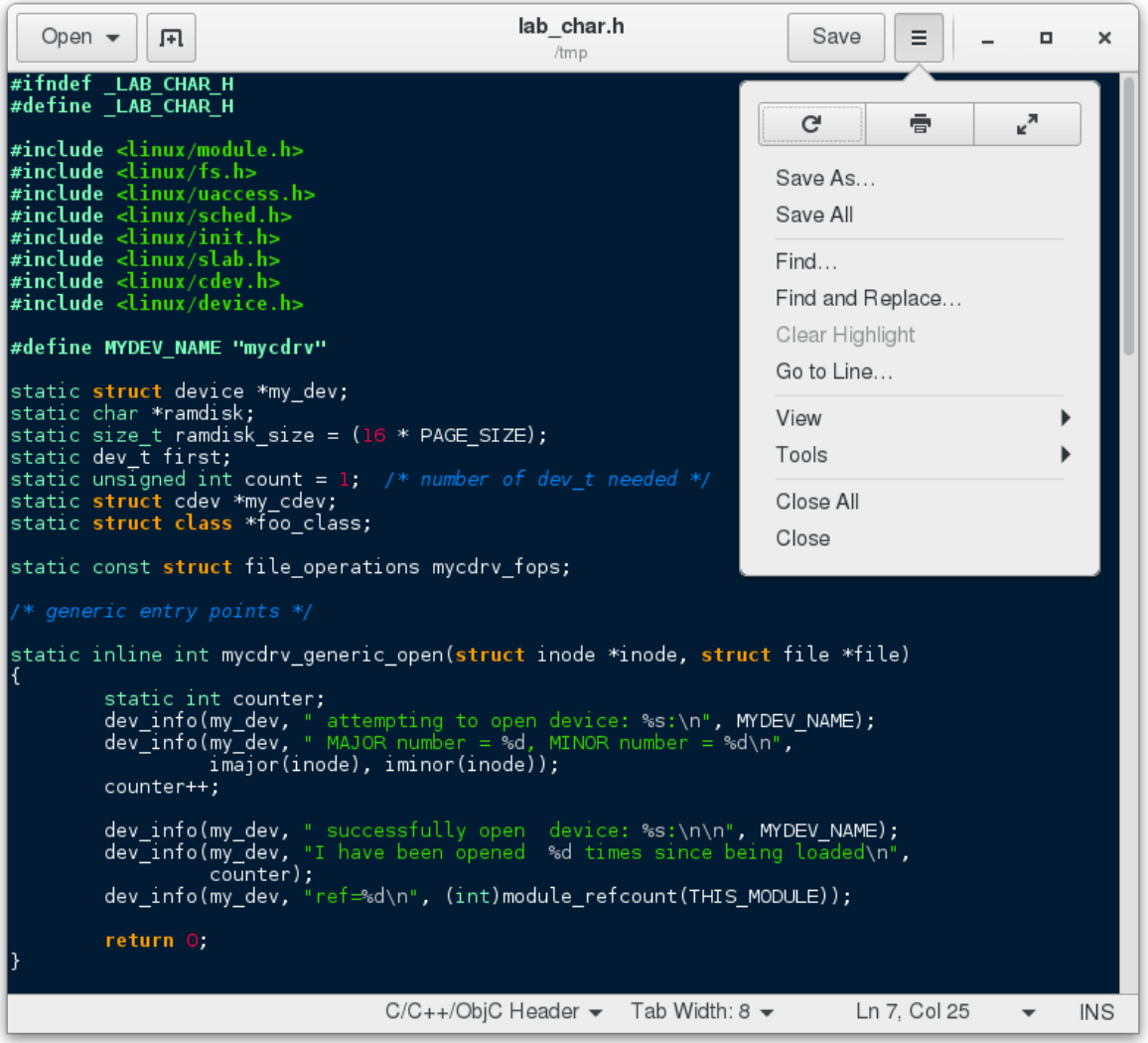
static inline int mycdrv_generic_open(struct inode *inode, struct file *file)
{
        static int counter;

        dev_info(my_dev, " attempting to open device: %s:\n", MYDEV_NAME);
        dev_info(my_dev, " MAJOR number = %d, MINOR number = %d\n",
                 imajor(inode), iminor(inode));
        counter++;

        dev_info(my_dev, " successfully open  device: %s:\n\n", MYDEV_NAME);
        dev_info(my_dev, "I have been opened  %d times since being loaded\n",
                 counter);
        dev_info(my_dev, "ref=%d\n", (int)module_refcount(THIS_MODULE));
                              [ Read 102 lines ]
^G Get Help  ^O Write Out ^W Where Is  ^K Cut Text  ^J Justify   ^C Cur Pos
^X Exit      ^R Read File ^\ Replace   ^U Uncut Text^T To Spell  ^_ Go To Line
```

gedit

- gedit (pronounced 'g-edit') is a simple-to-use graphical editor that can only be run within a Graphical Desktop environment.
- It is visually quite similar to the Notepad but is actually far more capable and very configurable and has a wealth of plugins available to extend its capabilities further.
- To open a new file, find the program in your desktop's menu system, or from the command line type `gedit <filename>`.
- If the file does not exist, it will be created.



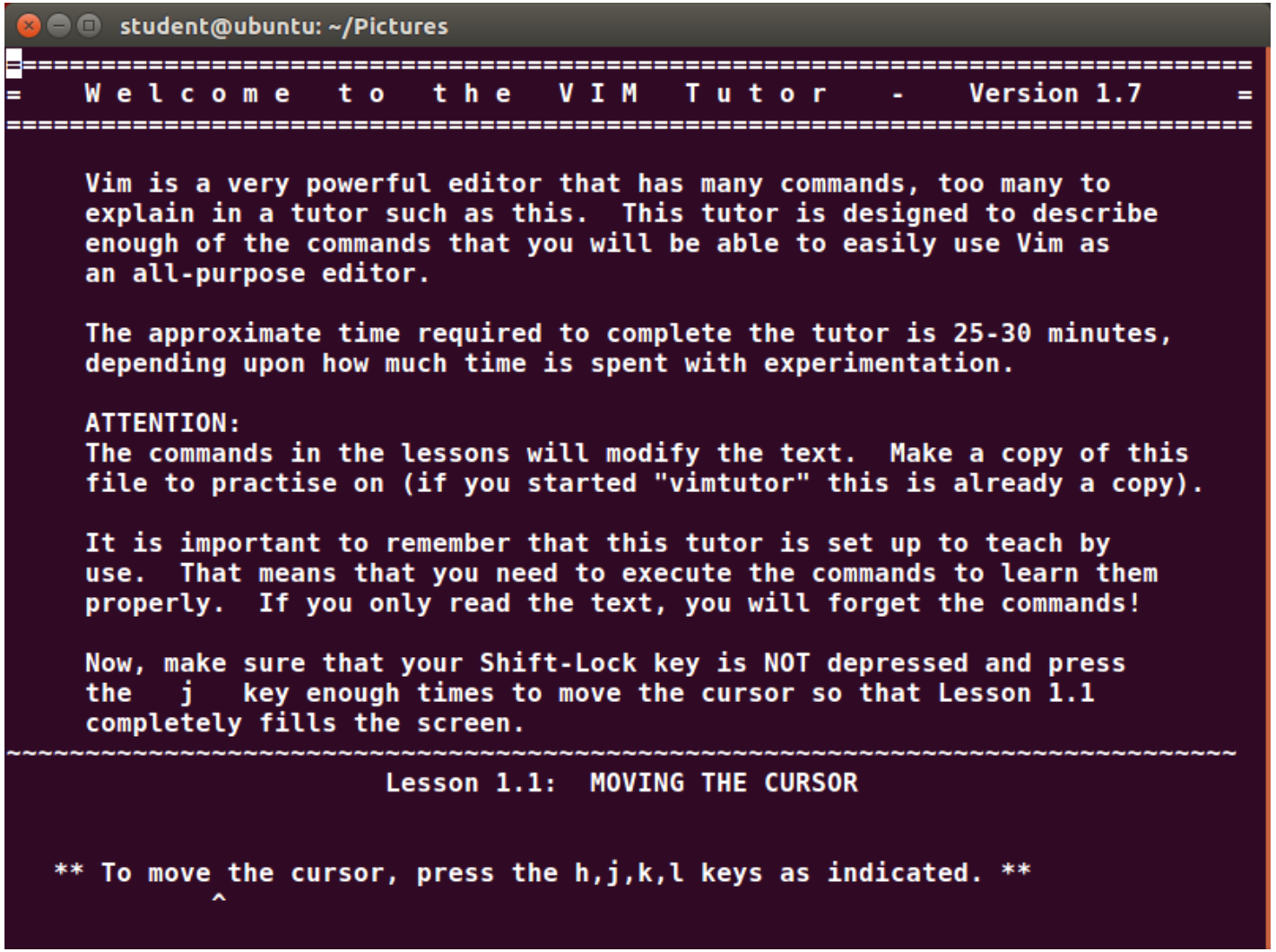
`vi` and `emacs`

- Developers and administrators experienced in working on UNIX-like systems almost always use one of the two venerable editing options: `vi` and `emacs`.
- Both are present or easily available on all distributions and are completely compatible with the versions available on other OSs.
- Both `vi` and `emacs` have a basic, purely text-based form that can run in a non-graphical environment.
  - They also have one or more graphical interface forms with extended capabilities; these may be friendlier for a less experienced user.
- While `vi` and `emacs` can have significantly steep learning curves for new users, they are extremely efficient when one has learned how to use them.

Introduction to `vi`

- The actual program installed on your system is vim, which stands for Vi IMproved and is aliased to the name vi.
- The name is pronounced as “vee-eye”.
- it is a standard tool installed on virtually all Linux distributions. Indeed, there may be times when there is no other editor available on the system.
- GNOME extends vi with a very graphical interface known as gvim and KDE offers kvim.
- When using vi, all commands are entered through the keyboard.

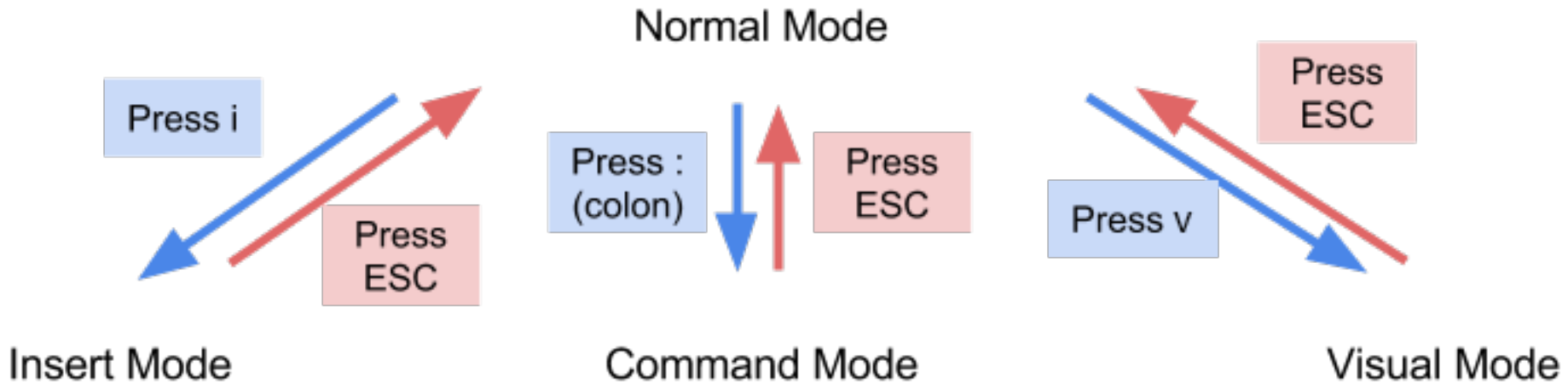
vimtutor


student@ubuntu: ~/Pictures
= W e l c o m e t o t h e V I M T u t o r - Version 1.7 =
Vim is a very powerful editor that has many commands, too many to
explain in a tutor such as this. This tutor is designed to describe
enough of the commands that you will be able to easily use Vim as
an all-purpose editor.
The approximate time required to complete the tutor is 25-30 minutes,
depending upon how much time is spent with experimentation.
ATTENTION:
The commands in the lessons will modify the text. Make a copy of this
file to practise on (if you started "vimtutor" this is already a copy).
It is important to remember that this tutor is set up to teach by
use. That means that you need to execute the commands to learn them
properly. If you only read the text, you will forget the commands!
Now, make sure that your Shift-Lock key is NOT depressed and press
the j key enough times to move the cursor so that Lesson 1.1
completely fills the screen.
Lesson 1.1: MOVING THE CURSOR
** To move the cursor, press the h,j,k,l keys as indicated. **

Modes in vi

Mode	Feature
Command	• By default, vi starts in Command mode. • Each key is an editor command. • Keyboard strokes are interpreted as commands that can modify file contents.
Insert	• Type `i` to switch to Insert mode from Command mode. • Insert mode is used to enter (insert) text into a file. • Insert mode is indicated by an "`? INSERT ?`" indicator at the bottom of the screen. • Press `Esc` to exit Insert mode and return to Command mode.
Line	• Type `:` to switch to the Line mode from Command mode. • Each key is an external command, including operations such as writing the file contents to disk or exiting. • Uses line editing commands inherited from older line editors. Most of these commands are actually no longer used. Some line editing commands are very powerful. • Press `Esc` to exit Line mode and return to Command mode.

Modes in vi



Visual mode	Usage
`v`	Normal visual mode
`V`	Visual line mode
`CTRL-v`	Visual block mode

Image: https://cs111.byu.edu/articles/vim/

Working with Files in vi

The table describes the most important commands used to start, exit, read, and write files in `vi`. The ENTER key needs to be pressed after all of these commands.

Command	Usage
`vi myfile`	Start the editor and edit `myfile`
`vi -r myfile`	Start and edit `myfile` in recovery mode from a system crash
`:r file2`	Read in `file2` and insert at current position
`:w`	Write to the file
`:w myfile`	Write out to `myfile`
`:w! file2`	Overwrite `file2`
`:x` or `:wq`	Exit and write out modified file
`:q`	Quit
`:q!`	Quit even though modifications have not been saved

Changing Cursor Positions in `vi`

- The table describes the most important keystrokes used when changing cursor position in vi.
- Line mode commands (those following colon :) require the ENTER key to be pressed after the command is typed.

Key	Usage
Arrow keys	To move up, down, left and right
`j` or `<ret>`	To move one line down
`k`	To move one line up
`h` or Backspace	To move one character left
`l` or Space	To move one character right
`0`	To move to beginning of line
`$`	To move to end of line

Key	Usage
`w`	To move to beginning of next word
`:0` or `1G`	To move to beginning of file
`:`*`n`* or *`n`*`G`	To move to line n
`:$` or `G`	To move to last line in file
Ctrl-F or Page Down	To move forward one page
Ctrl-B or Page Up	To move backward one page
`^l`	To refresh and center screen

Searching for Text in `vi`

- Commands Used to Search for Text:

Command	Usage
`/pattern`	Search forward for pattern
`?pattern`	Search backward for pattern

- Keystrokes Used to Search for Text:

Key	Usage
`n`	Move to next occurrence of search pattern
`N`	Move to previous occurrence of search pattern

- Find and replace :

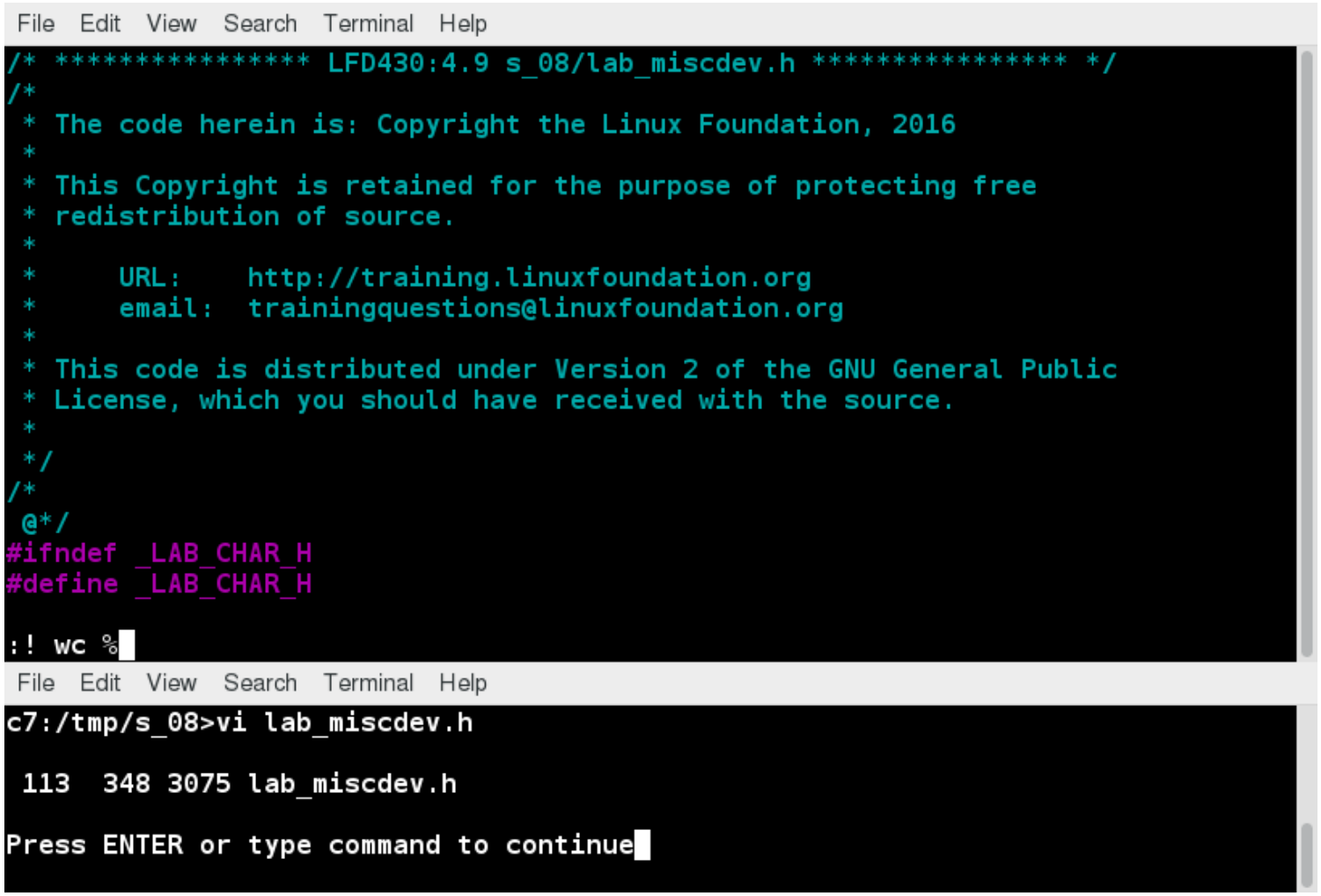
Command	Usage
`:%s/search_phrase/replace_phrase/g`	Replace *search_phrase* with *replace_phrase* for the whole file
`:s/search_phrase/replace_phrase/g`	Replace *search_phrase* with *replace_phrase* for the current line

Working with Text in `vi`

Key	Usage
`a`	Append text after cursor; stop upon Escape key
`A`	Append text at end of current line; stop upon Escape key
`i`	Insert text before cursor; stop upon Escape key
`I`	Insert text at beginning of current line; stop upon Escape key
`o`	Start a new line below current line, insert text there; stop upon Escape key
`O`	Start a new line above current line, insert text there; stop upon Escape key
`r`	Replace character at current position
`R`	Replace text starting with current position; stop upon Escape key

Key	Usage
`x`	Delete character at current position
N`x`	Delete *N* characters, starting at current position
`dw`	Delete the word at the current position
`D`	Delete the rest of the current line
`dd`	Delete the current line
N`dd` or `d`*N*`d`	Delete *N* lines
`u`	Undo the previous operation
`yy`	Yank (copy) the current line and put it in buffer
N`yy` or `y`*N*`y`	Yank (copy) *N* lines and put it in buffer
`P`	Paste at the current position the yanked line or lines from the buffer

Using External Commands in vi

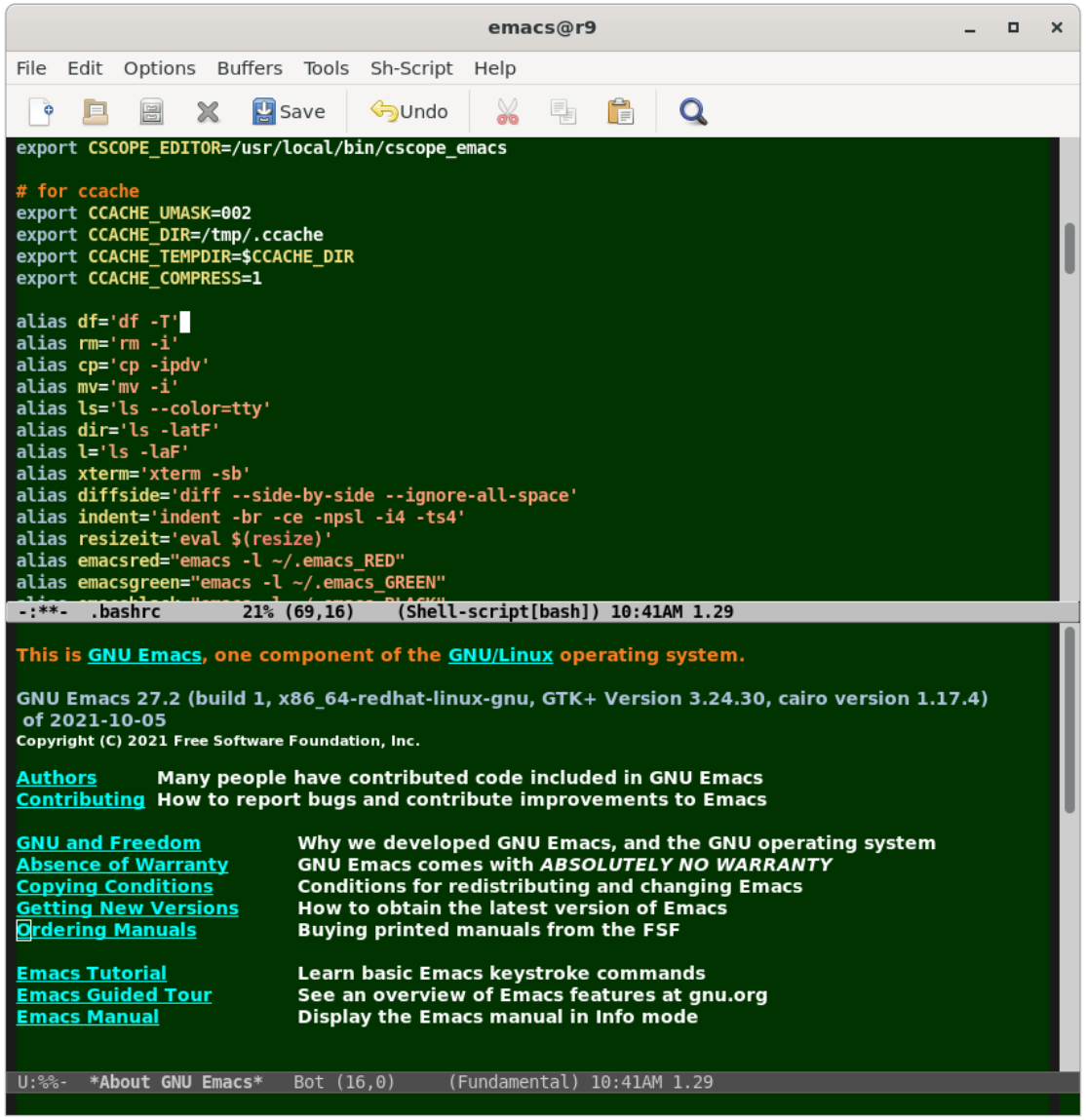


Introduction to emacs

- The emacs editor is a popular competitor for vi. Unlike vi, it does not work with modes.
- emacs is highly customizable and includes a large number of features.
- It was initially designed for use on a console but was soon adapted to work with a GUI as well. emacs has many other capabilities other than simple text editing.
  - For example, it can be used for email, debugging, etc.
- Rather than having different modes for command and insert, like vi, emacs uses the CTRL and Meta (Alt or Esc) keys for special commands.
- To install on Ubuntu

```
$ sudo apt update; sudo apt install emacs -y; emacs --version
```

Introduction to emacs



Working with emacs

Key	Usage
`emacs myfile`	Start emacs and edit `myfile`
`CTRL-x i`	Insert prompted for file at current position
`CTRL-x s`	Save all files
`CTRL-x CTRL-w`	Write to the file giving a new name when prompted
`CTRL-x CTRL-s`	Saves the current file
`CTRL-x CTRL-c`	Exit after being prompted to save any modified files

Changing Cursor Positions in emacs

Key	Usage
arrow keys	Use the arrow keys for up, down, left and right
CTRL-n	One line down
CTRL-p	One line up
CTRL-f	One character forward/right
CTRL-b	One character back/left
CTRL-a	Move to beginning of line
CTRL-e	Move to end of line
Meta-f	Move to beginning of next word

Key	Usage
Meta-b	Move back to beginning of preceding word
Meta-<	Move to beginning of file
Meta-g-g-n	Move to line n (can also use 'Esc-x Goto-line n')
Meta->	Move to end of file
CTRL-v or Page Down	Move forward one page
Meta-v or Page Up	Move backward one page
CTRL-l	Refresh and center screen

Searching for Text and Working with Text in emacs

Key	Usage
CTRL-s	Search forward for prompted pattern, or for next pattern
CTRL-r	Search backwards for prompted pattern, or for next pattern
CTRL-o	Insert a blank line
CTRL-d	Delete character at current position
CTRL-k	Delete the rest of the current line
CTRL-_	Undo the previous operation
CTRL- (space or CTRL-@)	Mark the beginning of the selected region (the end will be at the cursor position)
CTRL-w	Delete the current marked text and write it to the buffer
CTRL-y	Insert at current cursor location whatever was most recently deleted

Midterm Exam

- Computer-based exam (on Finex)
- Lecture slides are allowed, which are uploaded by the instructor
- Covers everything from Lecture and Lab 1 - 8.
- Cannot use any programs during the exam!